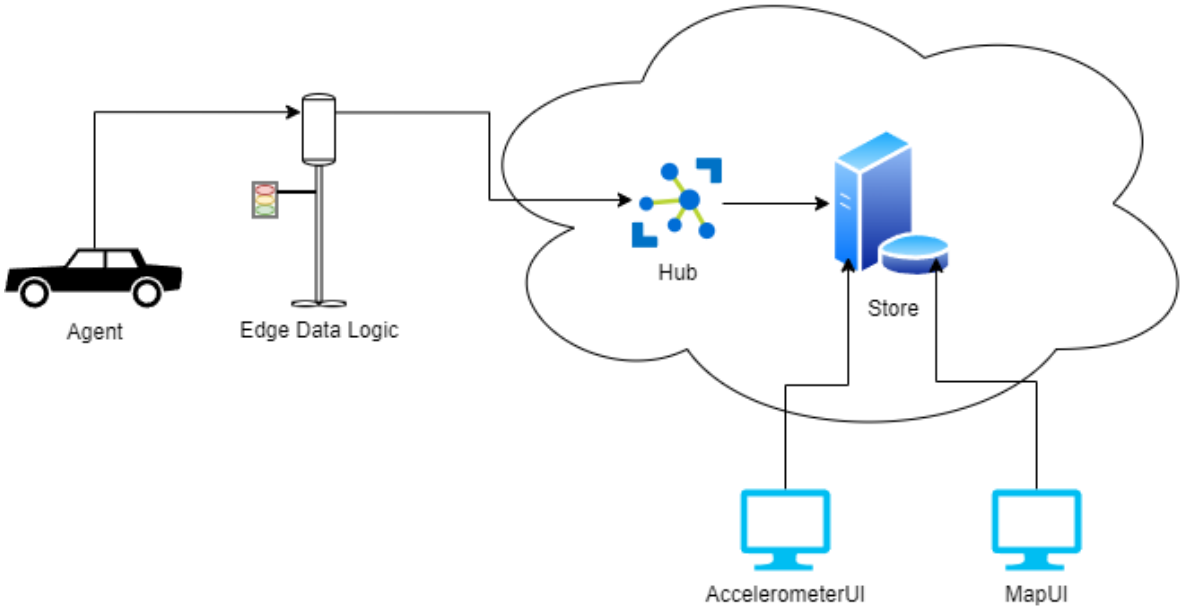
Лабораторна робота №1

Опис проекту



Завдання – розробити комплексну систему моніторингу та аналізу стану дорожнього покриття. Система може мати можливість збирати дані про різні параметри, такі як нерівності, тріщини, вибоїни та інші дефекти, які можуть вплинути на якість дороги. Зібрані дані слід проаналізувати, щоб визначити закономірності та тенденції, які можуть допомогти у прогнозуванні потреб у технічному обслуговуванні дорожньої мережі. Система також може мати можливість генерувати звіти та сповіщення для відповідних органів влади та/або водіїв.

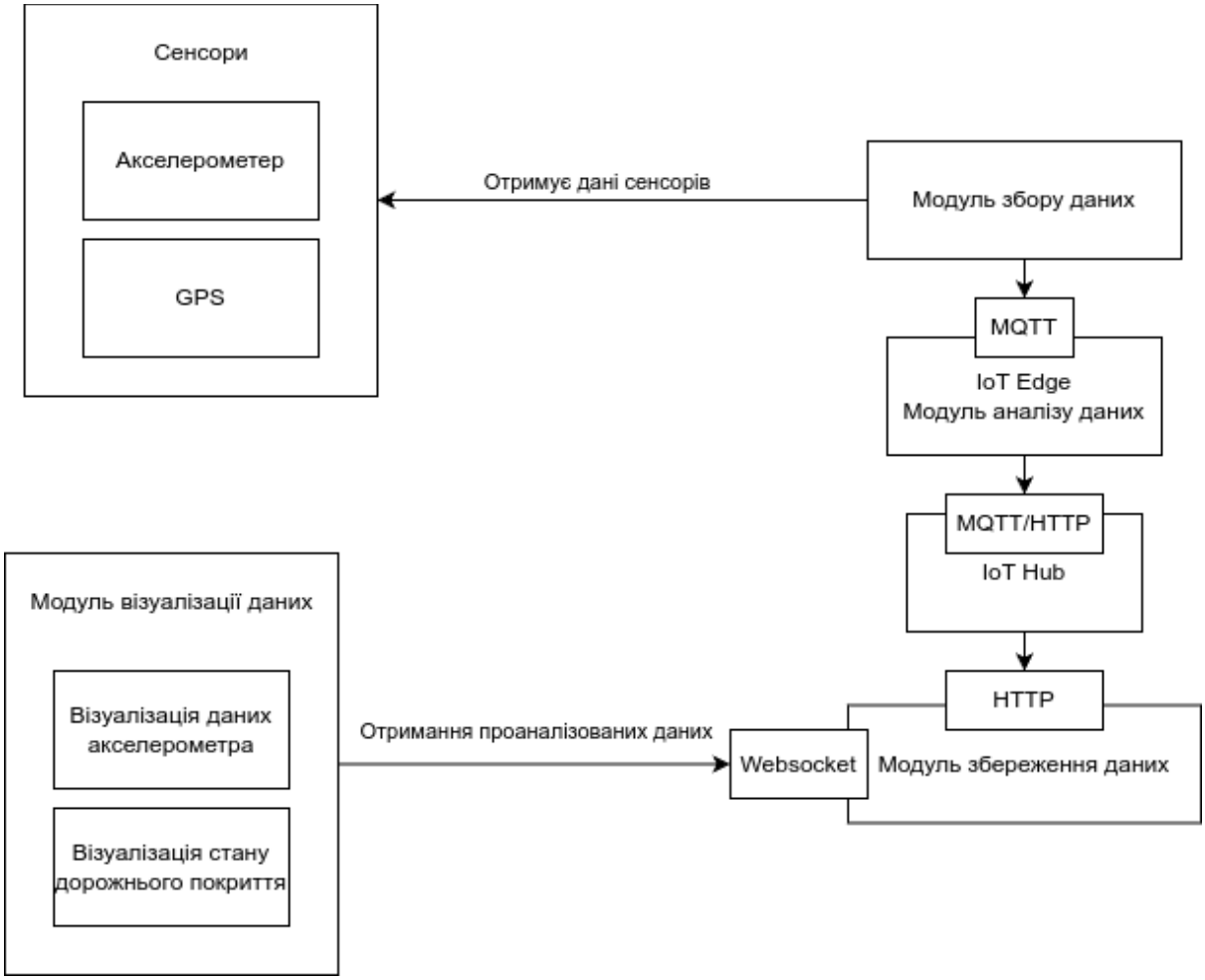
Метою цієї системи є підвищення безпеки дорожнього руху, зниження витрат на технічне обслуговування та покращення загального досвіду водіння для автомобілістів.

В систему входять пристрій на машині (stm32), edge пристої, хмарні або on-prem сервіси.

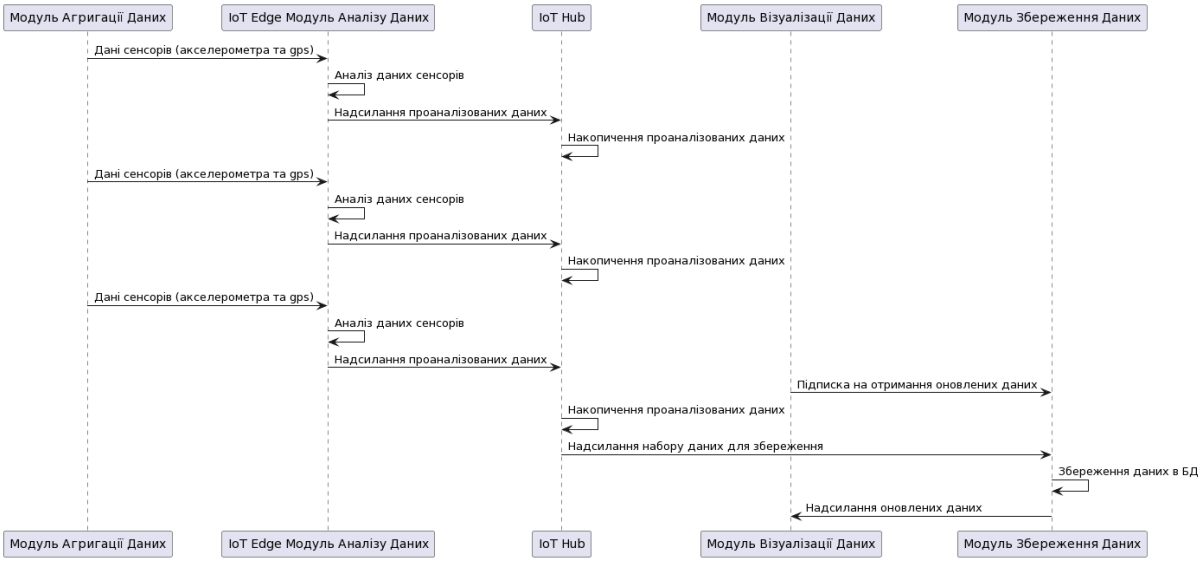
Система містити наступні частини (дозволяється розширювати):

- Модуль збору даних (Agent) - програма агент на машині, яка знімає показники сенсорів (наприклад раз в 100мс) та відправляє їх на сервер.

- Модуль первинної обробки даних (Edge Data Logic) - програма яка займається первинною обробкою даних, в нашому випадку визначає стан дорожнього покриття та відправляє проаналізовані дані на Hub.
- Модуль накопичення даних (Hub) - сервіс який займається накопиченням та аналізом (preprocessing) отриманих даних від Agent та подальшим зберіганням оброблених даних в БД. Перед збереженням діні потрібно накопичити (наприклад 10 записів), так як такі дані простіше аналізувати і запис в БД множини даних буде швидшим.
- Store - api для отримання та збереження даних в БД.
- Модуль візуалізації (Accelerometer UI, Map UI) - інтерфейс візуалізації (моніторингу) результатів системи.



Архітектура системи



Юзкейс роботи

Постановка задачі

Відбувається розробка програмного забезпечення, яке є частиною великої системи Розумного Міста. А саме розробка Agent частини, яка збирає дані отримані з датчиків вбудованих в сучасні машини для моніторингу стану дорожнього покриття. Проте не всі розробники маю доступ до датчиків для роботи з ними.

Я як розробник хочу мати змогу розробляти систему без фізичного пристою, датчиків (акселерометер) які видають інформацію.

Потрібнно абстрагуватися від конкретного джерела інформації або емулювати роботу датчиків.

Для рішення цієї задачі потрібно реалізувати програмну частину яка буде читати дані датчиків з файлу (до цього записаний csv file під назвою data.csv) та надсилати їх на Edge. В нашому випадку Agent та Edge будуть комунікувати через MQTT.

Кроки виконання

Надалі буде показана реалізація на python, проте ніхто не забороняє використовувати той технологічний стек який для вас є оптимальним.

До початку роботи потрібно встановити Docker та MQTT Explorer.

1. Створення віртуального середовища. Пакет **virtualenv** повинен бути встановлений в системі.

```
python -m venv venv
```

2. Активація середовища

```
source ./venv/bin/activate # linux
.\venv\Scripts\activate.bat # windows
```

3. Встановимо необхідні залежності(бібліотеки)

```
pip install paho-mqtt==1.6.1 # mqtt client
pip install marshmallow # for schema and converting object to json
```

4. Збережемо залежності в requirements.txt

```
pip freeze > requirements.txt
```

5. Структура проекту буде наступною

src
- *main.py*
- *config.py*
- *file_datasource.py*
- *schema*
  - *accelerometer_schema.py*
  - *gps_schema.py*
  - *aggregated_data_schema.py*
- *domain*
  - *accelerometer.py*
  - *gps.py*

aggregated_data.py

docker

mosquitto/config/mosquitto.conf

docker-compose.yaml

Dockerfile

requirements.txt

README.md

6. Реалізуємо моделі даних.

accelerometer.py

```
from dataclasses import dataclass


@dataclass
class Accelerometer:
    x: int
    y: int
    z: int
```

gps.py

```
from dataclasses import dataclass


@dataclass
class Gps:
    longitude: float
    latitude: float
```

aggregated_data.py

```python
from dataclasses import dataclass

from datetime import datetime
from domain.accelerometer import Accelerometer
from domain.gps import Gps


@dataclass
class AggregatedData:
    accelerometer: Accelerometer
    gps: Gps
    time: datetime
```

7. Наступним кроком реалізуємо схеми які будемо використовувати для конвертації об'єктів в json формат.

accelerometer_schema.py

```python
from marshmallow import Schema, fields
from domain.accelerometer import Accelerometer


class AccelerometerSchema(Schema):
    x = fields.Int()
    y = fields.Int()
    z = fields.Int()
```

location_schema.py

```python
from marshmallow import Schema, fields
from domain.gps import Gps


class GpsSchema(Schema):
    longitude = fields.Number()
    latitude = fields.Number()
```

aggregated_data_schema.py

```python
from marshmallow import Schema, fields
from schema.accelerometer_schema import AccelerometerSchema
from schema.gps_schema import GpsSchema
from domain.aggregated_data import AggregatedData


class AggregatedDataSchema(Schema):
    accelerometer = fields.Nested(AccelerometerSchema)
    gps = fields.Nested(GpsSchema)
    time = fields.DateTime('iso')
```

8. Реалізуємо файл конфігурацій

config.py

```python
import os


def try_parse(type, value: str):
    try:
        return type(value)
    except Exception:
        return None


# MQTT config
MQTT_BROKER_HOST = os.environ.get('MQTT_BROKER_HOST') or 'mqtt'
MQTT_BROKER_PORT = try_parse(int, os.environ.get('MQTT_BROKER_PORT')) or 1883
MQTT_TOPIC = os.environ.get('MQTT_TOPIC') or 'agent'

# Delay for sending data to mqtt in seconds
DELAY = try_parse(float, os.environ.get('DELAY')) or 1
```

9. Основну логіку, а саме file_datasource.py потрібно реалізувати самостійно. Перед читанням даних відкрити файли, при читанні зчитувати відповідний набір даних з файлів та повертати його.

file_datasource.py

```python
from csv import reader
from datetime import datetime
from domain.aggregated_data import AggregatedData


class FileDatasource:
    def __init__(self, accelerometer_filename: str, gps_filename: str) -> None:
        pass

    def read(self) -> AggregatedData:
        """Метод повертає дані отримані з датчиків"""

    def startReading(self, *args, **kwargs):
        """Метод повинен викликатись перед початком читання даних"""

    def stopReading(self, *args, **kwargs):
        """Метод повинен викликатись для закінчення читання даних"""
```

10.Реалізуємо основний файл для запуску програми main.py

```python
from paho.mqtt import client as mqtt_client
import json
import time
from schema.aggregated_data_schema import AggregatedDataSchema
from file_datasource import FileDatasource
import config

def connect_mqtt(broker, port):
    """Create MQTT client"""
    print(f"CONNECT TO {broker}:{port}")
    def on_connect(client, userdata, flags, rc):
        if rc == 0:
            print(f"Connected to MQTT Broker ({broker}:{port})!")
        else:
            print("Failed to connect {broker}:{port}, return code %d\n", rc)
            exit(rc)  # Stop execution
    client = mqtt_client.Client()
    client.on_connect = on_connect
    client.connect(broker, port)
    client.loop_start()
    return client

def publish(client, topic, datasource, delay):
    datasource.startReading()
    while True:
        time.sleep(delay)
        data = datasource.read()
        msg = AggregatedDataSchema().dumps(data)
        result = client.publish(topic, msg)
        # result: [0, 1]
        status = result[0]
        if status == 0:
            pass
            # print(f"Send `{msg}` to topic `{topic}`")
        else:
            print(f"Failed to send message to topic {topic}")

def run():
    # Prepare mqtt client
    client = connect_mqtt(config.MQTT_BROKER_HOST, config.MQTT_BROKER_PORT)
    # Prepare datasource
    datasource = FileDatasource("data/accelerometer.csv", "data/gps.csv")
    # Infinity publish data
    publish(client, config.MQTT_TOPIC, datasource, config.DELAY)

if __name__ == '__main__':
    run()
```

11. Добавимо Dockerfile для запуску даного агента в docker.

Dockerfile

```
# set base image (host OS)
FROM python:latest
# set the working directory in the container
WORKDIR /usr/agent
# copy the dependencies file to the working directory
COPY requirements.txt .
# install dependencies
RUN pip install -r requirements.txt
# copy the content of the local src directory to the working directory
COPY src/ .
# command to run on container start
CMD ["python", "main.py"]
```

12. Добавимо mosquitto.conf для запуску mqtt.

mosquitto.conf

```
persistence true
persistence_location /mosquitto/data/
listener 1883
## Authentication ##
allow_anonymous true
# allow_anonymous false
# password_file /mosquitto/config/password.txt
## Log ##
log_dest file /mosquitto/log/mosquitto.log
log_dest stdout
# listener 1883
```

13. Налаштуємо docker-compose.yaml для локального запуску та тестування програми.

docker-compose.yaml

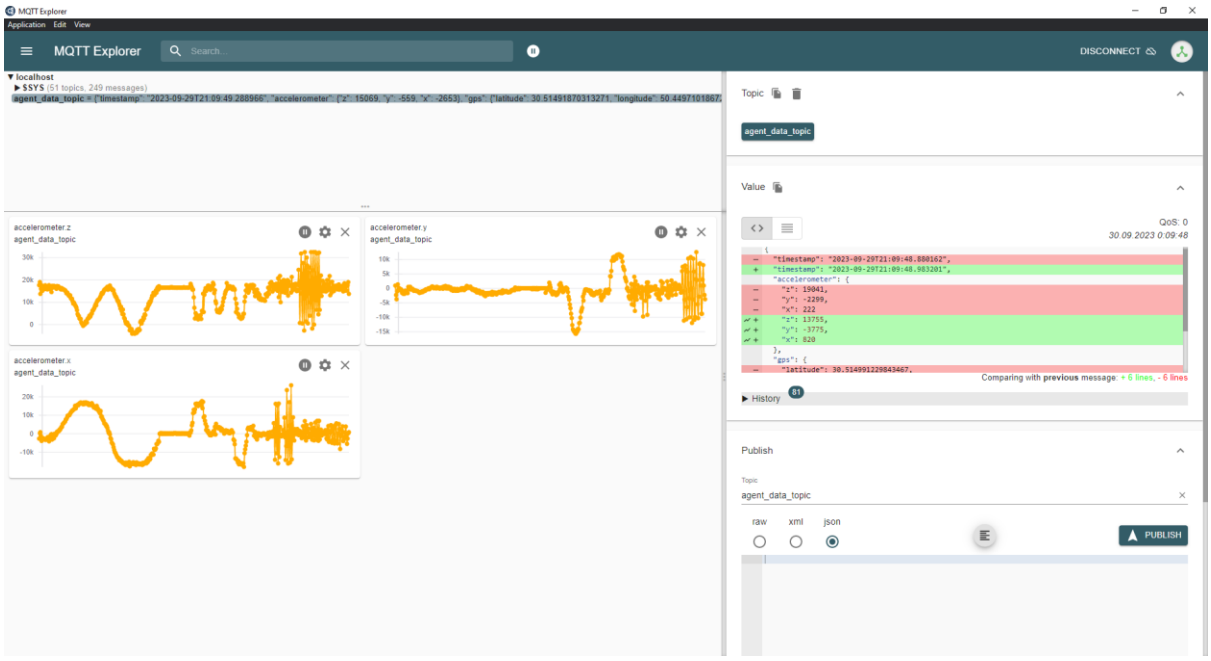
```yaml
version: "3.9"
name: "road_vision"
services:
  mqtt:
    image: eclipse-mosquitto
    container_name: mqtt
    volumes:
      - ./mosquitto:/mosquitto
      - ./mosquitto/data:/mosquitto/data
      - ./mosquitto/log:/mosquitto/log
    ports:
      - 1883:1883
      - 9001:9001
    networks:
      mqtt_network:

  fake_agent:
    container_name: agent
    build: ../
    depends_on:
      - mqtt
    environment:
      MQTT_BROKER_HOST: "mqtt"
      MQTT_BROKER_PORT: 1883
      MQTT_TOPIC: "agent_data_topic"
      DELAY: 0.1
    networks:
      mqtt_network:


networks:
  mqtt_network:
```

14. Тестові дані(data.csv, gps_data.csv) скачати за посиланням https://github.com/Toolf/project_template

15. Запуск програму та mqtt в docker

```
cd docker
docker-compose up --build
```

16. Перевірка працездатності в MQTT Explorer



Добавлення нового датчику в проєкт

1. Для додавання нових датчиків потрібно описати схему даних датчиків на агенті та об'єкт прикладної області, розглянемо на прикладі паркінгу

parking_schema.py

```
from agent.src.schema.gps_schema import GpsSchema
from marshmallow import Schema, fields


class ParkingSchema(Schema):
    empty_count = fields.Number()
    gps = fields.Nested(GpsSchema)
```

parking.py

```python
from agent.src.domain.gps import Gps


class Parking:
    empty_count: int
    gps: Gps
```

2. Добавити таблицю даних parking.csv
3. Розширити клас FileDatasource для читання даних з файлу parking_filename та відповідно розширити методи read, startReading, stopReading
4. Розширити логіку в main.py для відправки даних в відповідні топіки

Очікувані результати в протоколі

1. Посилання на git репозиторій з проектом
2. Реалізація FileDatasource
3. Показ результатів роботи в MQTT Explorer
4. Висновки

Ідеї для підвищення оцінки

Реалізація поступового читання (ф-ія read має при кожному звернені певну повертати набір даних, наприклад 5-7).

Реалізація нескінченного циклу читання з файлу (після доходження до кінця файлу починати з початку).

Реалізація Agent на пристої stm32 з реальними датчиками.